

Índice

- 1. Qué construimos vs qué ya existe
- 2. Principio de arquitectura: monolito modular primero
- 3. Stack tecnológico
- 4. Base de datos — schema completo corregido
- 5. Caché y colas — Redis 7
- 6. El diferenciador — LLM integrado
- 7. Stack frontend — Next.js + Portal
- 8. Cloud — GCP
- 9. Seguridad no negociable
- 10. Fases de construcción
- 11. Por qué vamos a ganar aunque llegemos últimos

01 / QUÉ CONSTRUIMOS VS QUÉ YA EXISTE

Procesamiento de tarjetas Visa/MC	MediaNet	Certificación PCI DSS, 19 años de operación
Clearing y liquidación interbancaria	MediaNet	Produbanco, Bolivariano, Internacional
Switch transaccional	MediaNet	Conexión con redes internacionales Visa/MC
Antifraude de red bancaria	MediaNet	Reglas a nivel de procesador
Diferidos y cuotas	MediaNet	Habilitados por banco adquirente
Infraestructura POS / datáfono	MediaNet	10,000+ comercios afiliados
API REST pública documentada	Construir	El corazón del nuevo producto
Portal self-service de comercios	Construir	Registro, dashboard, API keys, webhooks
Conector MediaNet (HTTP client)	Construir	Traduce la API pública al protocolo interno de MediaNet
Checkout embebable (JS widget)	Construir	2 líneas de script, modal de pago branded con soporte de cuotas

Link de Cobro	Construir	Link con monto fijo u opcional, vigencia, límite de usos
QR dinámico	Construir	QR generado desde el dashboard, compatible al instante
Cobro por Redes Sociales	Construir	Es el Link de Cobro compartido — no es arquitectura separada
Plugins WooCommerce / PrestaShop	Construir	Instalación en 5 minutos
Documentación interactiva + sandbox	Construir	Swagger UI incluido en FastAPI — gratis desde el día 1
Analytics con LLM	Construir	Insights automáticos en español — nadie lo tiene en Ecuador
Máquina de estados de transacciones	Construir	pending > processing > completed/failed
Sistema de webhooks firmados HMAC	Construir	Seguridad que ninguna pasarela local tiene

02 / PRINCIPIO DE ARQUITECTURA: MONOLITO MODULAR PRIMERO

La arquitectura v1.0 proponía 6 microservicios desde el día 1. Eso es un error para un MVP de 10 semanas.

Por qué microservicios en el MVP son un problema:

- Las primeras 4 semanas se van configurando infraestructura, no construyendo producto
- Cada servicio necesita su propio CI/CD, logging, health checks y comunicación entre servicios
- Depurar un bug que cruza 3 servicios toma 3x más tiempo que en un monolito
- No hay volumen que justifique la separación todavía

La decisión: monolito modular

Un solo proceso FastAPI organizado en módulos internos bien delimitados. La separación es lógica, no física. Cuando haya comercios reales con volumen, cada módulo se extrae como servicio independiente sin reescribir nada.

Estructura del proyecto:

```

medianetpay/
app/
main.py
config.py
modules/
payments/ – cobros, máquina de estados, idempotency
merchants/ – onboarding, API keys, configuración
connector/ – HTTP client hacia MediaNet (retry, circuit breaker)
links/ – Link de Cobro, QR dinámico
webhooks/ – webhooks salientes firmados HMAC
checkout/ – widget JS embebible
analytics/ – pipeline LLM, insights por comercio
api/v1/
charges.py
refunds.py
links.py
qr.py
merchants.py
webhooks.py
models/ – SQLAlchemy models
schemas/ – Pydantic schemas
middleware/ – auth, rate limiting, logging
utils/
worker.py – Celery worker
migrations/ – Alembic
tests/

```

Cuándo separar en microservicios: cuando el analytics necesite escalar independientemente, o cuando el volumen de webhooks requiera workers dedicados. Eso es fase 7+, no hoy.

03 / STACK TECNOLÓGICO

Framework	FastAPI 0.115+	ASGI async, Pydantic v2, Swagger UI gratis
Runtime	Python 3.12	async/await nativo, type hints estrictos
Servidor ASGI	Uvicorn + Gunicorn	Multi-worker en producción
Validación	Pydantic v2	Tipos estrictos, serialización automática
ORM	SQLAlchemy 2.0 async	Queries no bloqueantes, connection pool
Migrations	Alembic	Versionado de schema, rollback seguro
Driver DB	asyncpg	PostgreSQL async nativo
Task queue	Celery 5 + Redis	Cola de webhooks y reintentos

Jobs nocturnos	Cloud Scheduler + Cloud Run Job	Analytics nocturno — más simple que Celery Beat
HTTP client	httpx async	Conector MediaNet — retry, timeout, idempotencia
QR generation	qrcode[pil]	Genera PNG en el servidor
Frontend	Next.js 15 + TypeScript	SSR, App Router, RSC
Estilos	Tailwind CSS v4	Utility-first, dark mode nativo
Componentes	Shadcn/ui	Accesibles, personalizables
Data fetching	TanStack Query v5	Caché de estado, invalidación automática
Formularios	React Hook Form + Zod	Validación tipo-safe
Gráficos	Recharts	SVG responsivos para dashboard
Auth portal	NextAuth.js v5	JWT + refresh tokens, sesión en Redis
Docs API	Swagger UI (FastAPI built-in)	Gratis, sandbox en vivo desde el día 1
Checkout widget	Vanilla JS + iframe	2 líneas de script, sin dependencias
Plugins	PHP 8	WooCommerce y PrestaShop
LLM analytics	Anthropic API (Claude)	Insights en español por comercio
Linting	Ruff + Black	Formato consistente
Testing	pytest + httpx	Unit + integration tests

04 / BASE DE DATOS — SCHEMA COMPLETO

transactions:

```

id uuid PRIMARY KEY DEFAULT gen_random_uuid()
merchant_id uuid REFERENCES merchants(id) NOT NULL
amount numeric(12,2) NOT NULL
currency text NOT NULL DEFAULT 'USD'
status text NOT NULL DEFAULT 'pending'
-- 'pending' | 'processing' | 'completed' | 'failed' | 'reversed' | 'refunded'
payment_method text
-- 'card' | 'debit' | 'installments'
installments int DEFAULT 1
-- cuotas: 1=contado, 3, 6, 9, 12, 24 (diferidos Ecuador)
idempotency_key text UNIQUE NOT NULL
medianet_ref text
payment_link_id uuid REFERENCES payment_links(id)
description text
customer_email text
customer_name text
metadata jsonb
created_at timestampz DEFAULT now()
updated_at timestampz DEFAULT now()

```

merchants:

```

id uuid PRIMARY KEY DEFAULT gen_random_uuid()
business_name text NOT NULL
ruc text UNIQUE NOT NULL
email text UNIQUE NOT NULL
password_hash text NOT NULL
api_key_public text UNIQUE NOT NULL -- pk_live_xxx / pk_test_xxx
api_key_secret_hash text NOT NULL -- bcrypt de sk_live_xxx
webhook_url text
webhook_secret text
status text DEFAULT 'pending'
-- 'pending' | 'active' | 'suspended'
test_mode bool DEFAULT true
created_at timestampz DEFAULT now()
updated_at timestampz DEFAULT now()

```

payment_links:

```

id uuid PRIMARY KEY DEFAULT gen_random_uuid()
merchant_id uuid REFERENCES merchants(id) NOT NULL
token text UNIQUE NOT NULL -- URL: medianetpay.ec/pay/{token}
amount numeric(12,2) -- null = monto libre
currency text DEFAULT 'USD'
description text NOT NULL
expires_at timestampz -- null = no expira
max_uses int -- null = ilimitado
uses_count int DEFAULT 0
status text DEFAULT 'active'
-- 'active' | 'expired' | 'exhausted' | 'disabled'
qr_png_url text -- URL del QR en Cloud Storage
created_at timestampz DEFAULT now()

```

refunds:

```

id uuid PRIMARY KEY DEFAULT gen_random_uuid()
transaction_id uuid REFERENCES transactions(id) NOT NULL
merchant_id uuid REFERENCES merchants(id) NOT NULL
amount numeric(12,2) NOT NULL -- puede ser parcial
reason text
status text DEFAULT 'pending'
-- 'pending' | 'completed' | 'failed'
medianet_ref text
created_at timestamptz DEFAULT now()
updated_at timestamptz DEFAULT now()

```

webhook_events:

```

id uuid PRIMARY KEY DEFAULT gen_random_uuid()
merchant_id uuid REFERENCES merchants(id) NOT NULL
transaction_id uuid REFERENCES transactions(id)
event_type text NOT NULL
-- 'charge.completed' | 'charge.failed' | 'refund.completed' | 'charge.reversed'
payload jsonb NOT NULL
status text DEFAULT 'pending'
-- 'pending' | 'delivered' | 'failed'
attempts int DEFAULT 0
next_retry_at timestamptz
delivered_at timestamptz
created_at timestamptz DEFAULT now()

```

api_keys:

```

id uuid PRIMARY KEY DEFAULT gen_random_uuid()
merchant_id uuid REFERENCES merchants(id) NOT NULL
key_prefix text NOT NULL -- primeros 8 chars: pk_live_ab12
key_hash text NOT NULL -- bcrypt del key completo
type text NOT NULL -- 'public' | 'secret'
environment text NOT NULL DEFAULT 'test' -- 'test' | 'live'
is_active bool DEFAULT true
last_used_at timestamptz
created_at timestamptz DEFAULT now()

```

transaction_logs:

```

id uuid PRIMARY KEY DEFAULT gen_random_uuid()
transaction_id uuid REFERENCES transactions(id) NOT NULL
from_status text
to_status text NOT NULL
medianet_raw jsonb
triggered_by text -- 'api' | 'webhook_medianet' | 'system'
created_at timestamptz DEFAULT now()

```

analytics_snapshots:

```
id uuid PRIMARY KEY DEFAULT gen_random_uuid()
merchant_id uuid REFERENCES merchants(id) NOT NULL
period_start date NOT NULL
period_end date NOT NULL
total_volume numeric(14,2)
txn_count int
success_rate numeric(5,2)
top_hours jsonb
insights_text text -- output del LLM en español
generated_at timestamptz DEFAULT now()
```

Máquina de estados:

```
pending > processing > completed
> failed

completed > reversed > refunded
```

Reglas:

- Nunca actualizar status directamente — siempre a través de la máquina de estados
- Cada transición genera un registro inmutable en `transaction_logs`
- Índice `UNIQUE` en `idempotency_key` rechaza duplicados antes de llegar a MediaNet
- Webhooks duplicados de MediaNet son ignorados si la transición no tiene sentido

05 / CACHÉ Y COLAS — Redis 7

Rate limiting por API key:

Sliding window de 60 segundos. 100 req/min por defecto, configurable por comercio. HTTP 429 al exceder.

Caché de configuración de comercios:

Webhook URL, estado, configuración — cacheados 5 minutos. Evita una query a PostgreSQL en cada request de cobro.

Cola de webhooks salientes (Celery + Redis):

Los webhooks no se envían en el mismo request del cobro. Van a cola. Backoff exponencial: 10s > 30s > 2min > 10min > 1h. Hasta 5 reintentos. Si falla los 5, queda en estado failed visible en el dashboard.

Distributed locks para idempotencia:

El módulo de payments adquiere un lock en Redis con la `idempotency_key` antes de procesar. Si dos requests llegan simultáneamente con la misma clave, solo uno pasa.

Sesiones del portal:

JWT en Redis con TTL de 24h. Stateless en Cloud Run.

06 / EL DIFERENCIADOR — LLM Integrado

Cloud Scheduler dispara un Cloud Run Job cada noche a las 2am. El job agrega las transacciones de los últimos 30 días en PostgreSQL (queries directas — sin pandas en el MVP), llama a Claude con un prompt estructurado, y guarda el resultado en analytics_snapshots.

Ejemplo de output:

"Tus ventas caen 30% los lunes. Tus 3 mejores horas son 12-15h. Esta semana procesaste \$4,230 — 12% más que la semana anterior. El 18% de tus cobros fallidos son por fondos insuficientes — considera ofrecer diferidos a tus clientes."

Trigger	Cloud Scheduler	Dispara cada noche a las 2am
Ejecución	Cloud Run Job	Proceso efímero — no paga cuando no corre
Agregación	SQLAlchemy + PostgreSQL	Queries directas, sin pandas en el MVP
LLM call	Anthropic API (Claude)	Genera insights en español
Storage	analytics_snapshots	Persiste resultado
Entrega	Dashboard del portal	El comercio lo ve por la mañana

07 / STACK FRONTEND — Next.js + Portal

Framework	Next.js 15 + TypeScript	SSR, App Router, RSC
Estilos	Tailwind CSS v4	Utility-first, dark mode nativo
Componentes	Shadcn/ui	Accesibles, customizables
Data fetching	TanStack Query v5	Caché de estado, invalidación automática
Formularios	React Hook Form + Zod	Validación tipo-safe
Gráficos	Recharts	SVG responsivos para dashboard
Auth	NextAuth.js v5	JWT + refresh tokens, sesión en Redis
Docs API	Swagger UI (FastAPI built-in)	Gratis, sandbox en vivo desde el día 1
Checkout widget	Vanilla JS + iframe	2 líneas de script, sin React en el comercio
Plugins	PHP 8	WooCommerce y PrestaShop

Flujo de integración del developer:


```
// 1. Cargar el script
<script src="https://medianetpay.ec/v1/checkout.js"></script>

// 2. Inicializar con API key pública
const mp = MediaNetPay('pk_live_xxxx');

// 3. Abrir el modal
mp.checkout({
  amount: 5000,
  currency: 'USD',
  installments: [1, 3, 6, 12],
  onSuccess: (txn) => console.log('Pagado:', txn.id),
  onError: (err) => console.error(err)
});
```

08 / CLOUD — GCP

Cloud Run	API monolito + Celery worker	\$0-50	Escala a cero
Cloud Run Job	Analytics nocturno	\$0	Solo paga mientras corre (~2 min/noche)
Cloud SQL PostgreSQL 16	Base de datos managed	\$50-100	Backups automáticos, failover
Memorystore Redis 7	Caché + cola Celery	\$30-50	Managed, sin operaciones
Cloud Storage	QR PNGs, assets	\$0-5	QR generados
Secret Manager	Credenciales, secrets	\$0	Rotación sin redeploy
Cloud Build + Artifact Registry	CI/CD	\$0-10	Build y deploy automático
Cloud Monitoring + Logging	Observabilidad	\$0-20	Alertas, traces, logs
Cloud Armor	WAF + DDoS	\$5-20	Obligatorio en fintech
Cloud Scheduler	Trigger analytics	\$0	3 jobs gratis siempre
Load Balancer	HTTPS y routing	\$20	TLS termination

Total estimado mes 1-3: \$150-280/mes.

Con Google for Startups Cloud Program: hasta \$200,000 en créditos.

09 / SEGURIDAD NO NEGOCIABLE

CRÍTICO — Nunca PANs en nuestro sistema:

La API recibe tokens de MediaNet, no números de tarjeta crudos. Esto nos pone en SAQ A (PCI DSS más liviano).

CRÍTICO — API keys doble capa:

pk_live_xxx — pública, solo inicializa el checkout en el frontend del comercio.

sk_live_xxx — secreta, autoriza cobros desde el servidor del comercio, hasheada con bcrypt.

CRÍTICO — Idempotency keys:

UUID único por intento de cobro. Índice UNIQUE en DB + lock en Redis. Si la red falla y el comercio reintenta 3 veces, se cobra una sola vez.

ALTO — Firma de webhooks HMAC-SHA256:

Header X-MediaNetPay-Signature: sha256=xxxxx en cada webhook saliente.

Sin esto, cualquiera puede mandar un webhook falso diciendo "cobro exitoso".

ALTO — Rate limiting por API key:

100 req/min por defecto, configurable. HTTP 429 al exceder.

ALTO — HTTPS everywhere + HSTS:

TLS 1.3 obligatorio. Strict-Transport-Security: max-age=31536000.

MEDIO — Secrets en GCP Secret Manager:

Nunca en el código ni en los logs.

MEDIO — Audit logs inmutables:

Cada cambio de estado en transaction_logs. No se borra. Esencial para disputas con bancos.

10 / FASES DE CONSTRUCCIÓN

FASE 1 — Semanas 1-2: Conector MediaNet + Infraestructura base

Objetivo: cobro real de punta a punta desde código Python.

- Entender el protocolo interno de MediaNet (endpoint, auth, formato de request/response)
- Construir modules/connector/ con httpx async, retry (3 intentos), circuit breaker y logging
- Configurar GCP: Cloud Run, Cloud SQL, Memorystore, Secret Manager, Cloud Storage
- Alembic + migrations iniciales: merchants, transactions, transaction_logs
- Primer cobro en sandbox de MediaNet guardado en PostgreSQL

Entregable: script Python que hace un cobro real contra el sandbox de MediaNet y lo guarda en la DB.

FASE 2 — Semanas 3-4: API core pública + Link de Cobro + QR

Objetivo: Postman collection completa. Link de Cobro y QR funcionando. Swagger UI disponible.

Endpoints a construir:

```
POST /v1/charges – crear cobro
GET /v1/charges/{id} – consultar cobro
POST /v1/refunds – crear reembolso (parcial o total)
GET /v1/refunds/{id} – consultar reembolso
POST /v1/links – crear Link de Cobro
GET /v1/links/{token} – página pública del link
POST /v1/qr – generar QR PNG
GET /v1/webhooks/test – disparar webhook de prueba
```

- Autenticación con API keys (pk_ / sk_)
- Máquina de estados + transaction_logs
- Idempotency keys + distributed lock en Redis
- Rate limiting por API key
- Tabla payment_links — vigencia, monto libre u obligatorio, límite de usos
- Tabla refunds
- QR PNG con qrcode[pil] > Cloud Storage > URL pública
- Cola de webhooks salientes con Celery (HMAC-SHA256)
- Campo installments en transactions — opciones configurables por comercio

Entregable: un developer externo puede hacer un cobro, un reembolso, generar un Link de Cobro y un QR usando solo Swagger. Sin llamar a nadie.

FASE 3 — Semanas 5-6: Portal self-service de comercios

Objetivo: un comercio se registra, obtiene API keys y ve sus cobros sin asistencia humana.

- Registro con RUC + validación
- Generación automática de API keys (test y live)
- Dashboard de transacciones en tiempo real
- Detalle de transacción con transaction_logs
- Vista de reembolsos
- Gestión de Links de Cobro y QR (crear, ver usos, deshabilitar)
- Configuración de webhook URL + botón de prueba
- Deploy en medianetpay.ec

Entregable: onboarding completo en menos de 10 minutos sin asistencia humana.

FASE 4 — Semanas 7-8: Checkout widget + Plugins + Documentación

Objetivo: un developer integra en menos de 30 minutos sin abrir un ticket.

- Checkout widget JS embebable (Vanilla JS + iframe)
- Modal branded con campos de tarjeta
- Soporte de cuotas/diferidos (opciones configuradas por el comercio)
- Callbacks onSuccess / onError / onCancel
 - Plugin WooCommerce (PHP 8)
- Instalable desde WP Admin
- Configuración visual de API keys y cuotas
- Toggle test / live

- Plugin PrestaShop (PHP 8) — mismo patrón
- Documentación narrativa enriquecida con ejemplos en curl, Python, PHP, JS

Entregable: developer con WooCommerce integra MediaNetPay en menos de 30 minutos sin soporte.

FASE 5 — Semanas 9-10: Beta privada con comercios reales

Objetivo: primeras transacciones de dinero real.

- Onboarding de 5-10 comercios beta seleccionados
- Activar modo live para los comercios beta
- Cloud Monitoring dashboards: latencia del conector, tasa de éxito, queue de webhooks
- Alertas: error rate > 1%, latencia > 2s, cola de webhooks > 100 items
- Ajustes de UX y performance basados en feedback real
- Runbook de incidentes documentado

Entregable: primeras transacciones reales completadas. Métricas de baseline establecidas.

FASE 6 — Semanas 11-12: Analytics LLM + Lanzamiento público

Objetivo: producto completo en producción, abierto a todos los comercios de MediaNet.

- Analytics pipeline: Cloud Scheduler > Cloud Run Job > Claude API > analytics_snapshots
- Dashboard de insights en el portal del comercio
- Alertas de anomalías por email
- Campaña de lanzamiento hacia la base de 10,000+ comercios de MediaNet
- Página de status pública (status.medianetpay.ec)

Entregable: producto en producción abierto al mercado. Primeras métricas de adopción.

FASE 7 — Post-lanzamiento: Iteración y escala

Basado en métricas de uso real:

- Chat con tus datos (agente LLM con contexto de transacciones del comercio)
 - Exportación de reportes en CSV/Excel desde el dashboard
 - Plugin Magento si hay demanda
 - Separar analytics como servicio independiente si el volumen lo requiere
 - Multi-moneda si hay demanda de comercios internacionales
-

11 / POR QUÉ VAMOS A GANAR AUNQUE LLEGUEMOS ÚLTIMOS

Datafast lleva 20 años. Kushki tiene VC. Nosotros partimos con los acuerdos bancarios resueltos, el canal de 10,000 comercios listo, y construimos el producto que ninguno de los dos se molestó en hacer bien.

Developer experience nivel Stripe — el primero en Ecuador.

Registro > API key > sandbox > primer cobro: 10 minutos. Sin formularios en papel, sin \$80 de certificación, sin \$150/mes de mensualidad mínima.

Sin barrera de entrada.

Una tienda con 30 ventas/mes no puede pagar \$150/mes. Nosotros cobramos 0 hasta que el comercio cobre.

LLM integrado — el moat que no se copia rápido.

Kushki y Datafast tienen cultura bancaria. No tienen expertise en LLMs. Nosotros convertimos la pasarela en un producto de valor, no solo un registro de transacciones.

El mercado ecuatoriano nunca tuvo un Stripe. Los ingredientes siempre estuvieron en MediaNet. Solo faltaba alguien que construyera la capa de producto encima.

MediaNetPay | Documento Técnico Interno v2.0 | Confidencial | Mayo 2026